what suggestions do you have to refactor this component?

```
import { AlertTriangle } from "lucide-react";
import { useState } from "react";
import { toast } from "react-hot-toast";

import { AdvancedOptionToggle } from
"../../AdvancedOptionToggle";
import { Button } from "../../Button";
import { Input } from "../../Input";
import { Label } from "../../Label";
import {
  checkForYoutubePrivacyMode,
  checkForYoutubeUrl,
  extractYoutubeId,
  parseVideoUrl,
} from "../lib/utils";

interface VideoSettingsProps {
  uploadedVideoUrl: string;
  setUploadedVideoUrl: (videoUrl: string) => void;
  onFileUpload: (uploadedUrl: string[] | undefined,
fileType: "image" | "video") => void;
  videoUrl: string;
  setVideoUrlTemp: (videoUrl: string) => void;
}

export const VideoSettings = ({
  uploadedVideoUrl,
  setUploadedVideoUrl,
  onFileUpload,
  videoUrl,
  setVideoUrlTemp,
}: VideoSettingsProps) => {
  const [isYoutubeLink, setIsYoutubeLink] =
useState(checkForYoutubeUrl(uploadedVideoUrl));
```

```
  const [isYoutubePrivacyModeEnabled,
setIsYoutubePrivacyModeEnabled] = useState(
    checkForYoutubePrivacyMode(uploadedVideoUrl)
  );
  const [showPlatformWarning, setShowPlatformWarning]
= useState(false);
  const toggleYoutubePrivacyMode = () => {

setIsYoutubePrivacyModeEnabled(!isYoutubePrivacyMod
eEnabled);

    const videoId = extractYoutubeId(uploadedVideoUrl);
    if (!videoId) {
      toast.error("Invalid YouTube URL");
      return;
    }
    const newUrl = isYoutubePrivacyModeEnabled
      ? `https://www.youtube.com/embed/${videoId}`
      : `https://www.youtube-
nocookie.com/embed/${videoId}`;

    setUploadedVideoUrl(newUrl);
    if (videoUrl) {
      onFileUpload([newUrl], "video");
    }
  };

  const handleAddVideo = (e:
React.MouseEvent<HTMLElement, MouseEvent>) => {
    e.preventDefault();
    const parsedUrl = parseVideoUrl(uploadedVideoUrl);
    if (parsedUrl) {
      setUploadedVideoUrl(parsedUrl);
      onFileUpload([parsedUrl], "video");
    } else {
      toast.error("Url not supported");
    }
  };

  const handleRemoveVideo = (e:
```

```
React.MouseEvent<HTMLElement, MouseEvent>) => {
   e.preventDefault();
   setVideoUrlTemp("");
   onFileUpload([], "video");
  };

  const handleVideoUrlChange = (e:
React.ChangeEvent<HTMLInputElement>) => {
   const videoUrl = e.target.value;
   setUploadedVideoUrl(videoUrl);
   // Check if the URL is from one of the supported
platforms
   const isSupportedPlatform = [
    "youtube.com",
    "youtu.be",
    "youtube-nocookie.com",
    "vimeo.com",
    "loom.com",
   ].some((domain) => videoUrl.includes(domain));

   setIsYoutubeLink(checkForYoutubeUrl(videoUrl));
   setShowPlatformWarning(!isSupportedPlatform);

setIsYoutubePrivacyModeEnabled(checkForYoutubePriva
cyMode(videoUrl));
  };

  const isAddButtonDisabled = () => {
   return uploadedVideoUrl.trim() !== "" ? false : true;
  };

  return (
   <form className="flex flex-col space-y-4">
    <Label>Video URL(Youtube, Vimeo or Loom)</Label>
    <div className="flex h-10 items-center space-x-2">
     <Input
      className="w-full"

placeholder="https://www.youtube.com/embed/VIDEO_I
D"
```

```
        value={uploadedVideoUrl}
        onChange={(e) => handleVideoUrlChange(e)}
       />
       {uploadedVideoUrl && videoUrl ===
uploadedVideoUrl ? (
        <Button variant="darkCTA" onClick={(e) =>
handleRemoveVideo(e)}>
         Remove
        </Button>
       ) : (
        <Button variant="darkCTA" onClick={(e) =>
handleAddVideo(e)} disabled={isAddButtonDisabled()}>
         Add
        </Button>
       )}
      </div>

      {showPlatformWarning && (
       <div className="flex items-center space-x-2
rounded-md border bg-slate-100 p-2 text-sm">
        <AlertTriangle className="h-4 w-4" />
        <p>Please enter a valid Youtube, Vimeo or loom
Url</p>
       </div>
      )}
      {isYoutubeLink && (
       <AdvancedOptionToggle
        htmlId="closeOnNumberOfResponse"
        isChecked={isYoutubePrivacyModeEnabled}
        onToggle={() => {
         toggleYoutubePrivacyMode();
        }}
        title="YouTube Privacy Mode"
        description="Keeps user tracking to a minimum"
        childBorder={true}></AdvancedOptionToggle>
      )}
    </form>
  );
};
```

Your `VideoSettings` component is well-organized and functional, but there are opportunities to refine it further for better readability, maintainability, and adherence to best practices. Here are my suggestions for refactoring:

1. **Extract Utility Functions**: The functions within your component, like `toggleYoutubePrivacyMode`, `handleAddVideo`, `handleRemoveVideo`, and `handleVideoUrlChange`, could be extracted to a separate utility or service file. This would make your component cleaner and focus primarily on rendering logic.
2. **Custom Hook for State Logic**: The logic for handling state changes and validations (e.g., checking YouTube URL, privacy mode, etc.) can be abstracted into a custom hook. This will make the component easier to read and allow reuse of the state logic in other components if needed.
3. **Use `useCallback` for Event Handlers**: To optimize performance, wrap event handlers with `useCallback`. This ensures that these functions are not recreated on every render, which is particularly useful if they are passed as props to child components.
4. **Simplify Conditional Rendering**: Your conditionals for rendering the `Button` and warning messages can be simplified for better readability.

Here's a refactored version of your component with these suggestions in mind:

tsx Copy code

```tsx
// VideoSettings.tsx
import React, { useState, useCallback } from 'react';
import { AlertTriangle } from 'lucide-react';
import { toast } from 'react-hot-toast';
import { Input, Button, Label, AdvancedOptionToggle } from '../../componen
import { useVideoSettings } from '../hooks/useVideoSettings';

interface VideoSettingsProps {
  uploadedVideoUrl: string;
  setUploadedVideoUrl: (videoUrl: string) => void;
  onFileUpload: (uploadedUrl: string[] | undefined, fileType: 'image' | 'v
  videoUrl: string;
  setVideoUrlTemp: (videoUrl: string) => void;
}

export const VideoSettings: React.FC<VideoSettingsProps> = ({
```

```
    uploadedVideoUrl,
    setUploadedVideoUrl,
    onFileUpload,
    videoUrl,
    setVideoUrlTemp,
  }) => {
    const {
      isYoutubeLink,
      isYoutubePrivacyModeEnabled,
      showPlatformWarning,
      handleAddVideo,
      handleRemoveVideo,
      handleVideoUrlChange,
      toggleYoutubePrivacyMode,
      isAddButtonDisabled,
    } = useVideoSettings(uploadedVideoUrl, setUploadedVideoUrl, onFileUpload,

    return (
      <form className="flex flex-col space-y-4">
        <Label>Video URL (YouTube, Vimeo, or Loom)</Label>
        <div className="flex h-10 items-center space-x-2">
          <Input
            className="w-full"
            placeholder="https://www.youtube.com/embed/VIDEO_ID"
            value={uploadedVideoUrl}
            onChange={handleVideoUrlChange}
          />
          {uploadedVideoUrl && videoUrl === uploadedVideoUrl ? (
            <Button variant="darkCTA" onClick={handleRemoveVideo}>
              Remove
            </Button>
          ) : (
            <Button variant="darkCTA" onClick={handleAddVideo} disabled={isA
              Add
            </Button>
          )}
        </div>
        {showPlatformWarning && (
          <div className="flex items-center space-x-2 rounded-md border bg-s
            <AlertTriangle className="h-4 w-4" />
```

```
            <p>Please enter a valid YouTube, Vimeo, or Loom URL</p>
          </div>
        )}
        {isYoutubeLink && (
          <AdvancedOptionToggle
            htmlId="youtubePrivacyMode"
            isChecked={isYoutubePrivacyModeEnabled}
            onToggle={toggleYoutubePrivacyMode}
            title="YouTube Privacy Mode"
            description="Keeps user tracking to a minimum"
            childBorder={true}
          />
        )}
      </form>
    );
  };
```

The `useVideoSettings` custom hook (not shown here) would encapsulate all state logic and handlers. Remember, for the custom hook and any utility functions, you'll need to consider their implementation based on your current logic and possibly adjust them to fit into the new structure.

By following these suggestions, your component becomes more focused on rendering, and its logic becomes more reusable and easier to test.